AYUDANTÍA I1

JUAN JOSÉ MERUANE
ARTURO PACHECO

ARQUITECTURA CLIENTE SERVIDOR

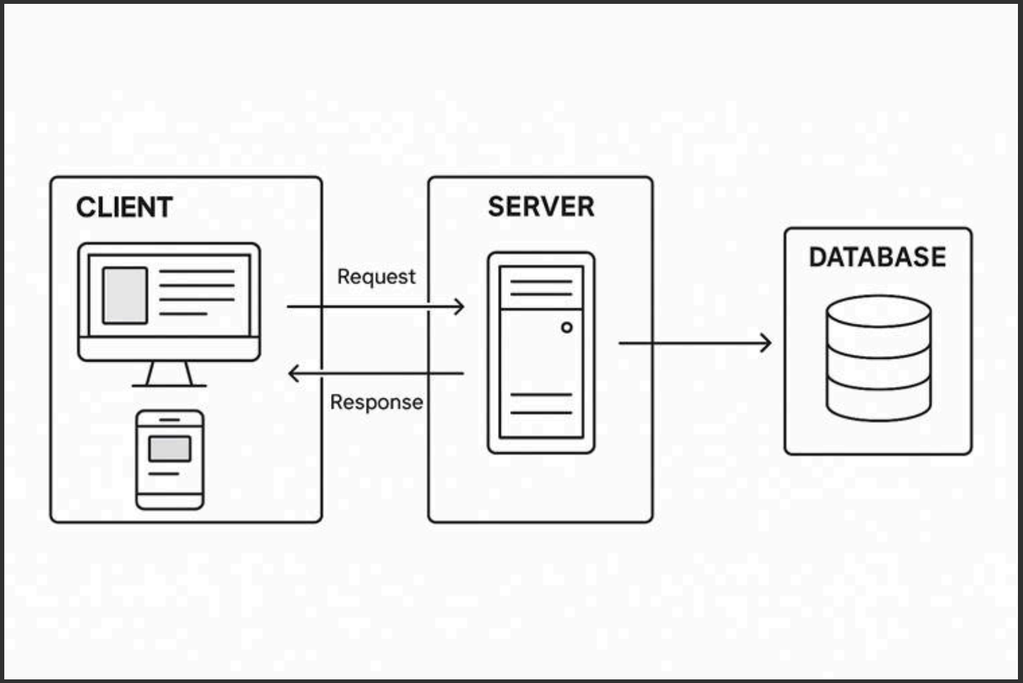
La Web sigue un modelo **cliente-servidor**

- **Cliente** (browser) → envía HTTP requests
- **Servidor** → responde con recursos (HTML, datos, servicios)
- Front-end (cliente) → interfaz y experiencia
- Back-end (Servidor) → lógica de negocio y datos

Componentes físicos esenciales de una Web App

- Computador cliente
- Navegador (browser)
- Conexión a internet
- Servidor web

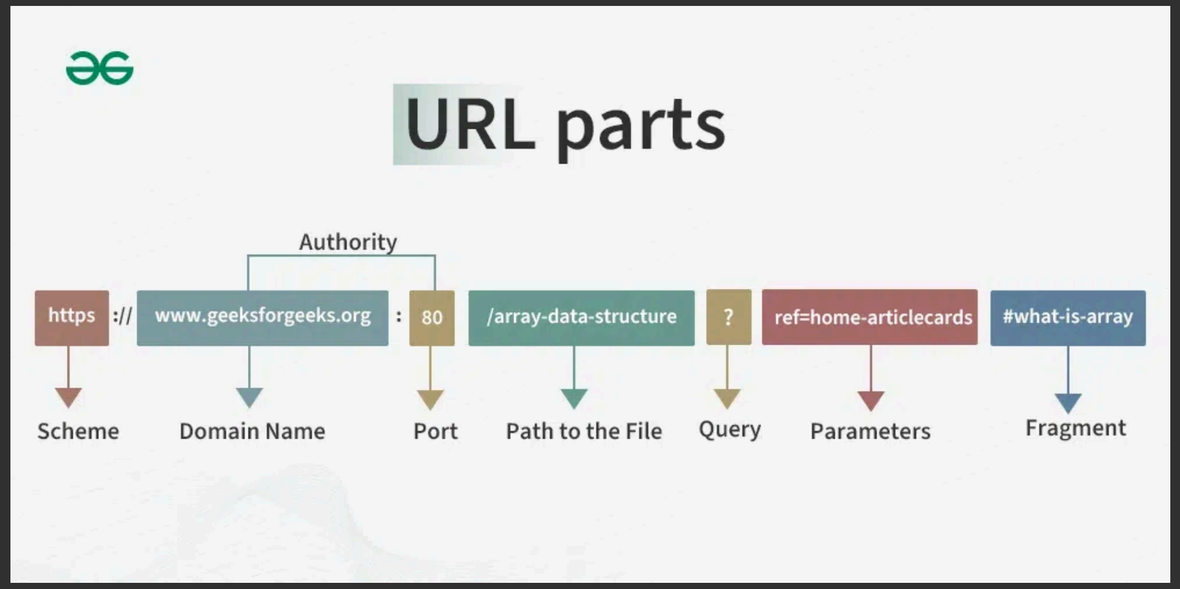
El navegador solicita recursos y el servidor responde. Hoy los navegadores son complejos e interpretan HTML, CSS y JavaScript.



URI/URL Y LA UBICACIÓN DE RECURSOS

URI → identificador único de recursos

URL → tipo de URI que indica cómo localizar un recurso en la web



DNS (Domain Name System):

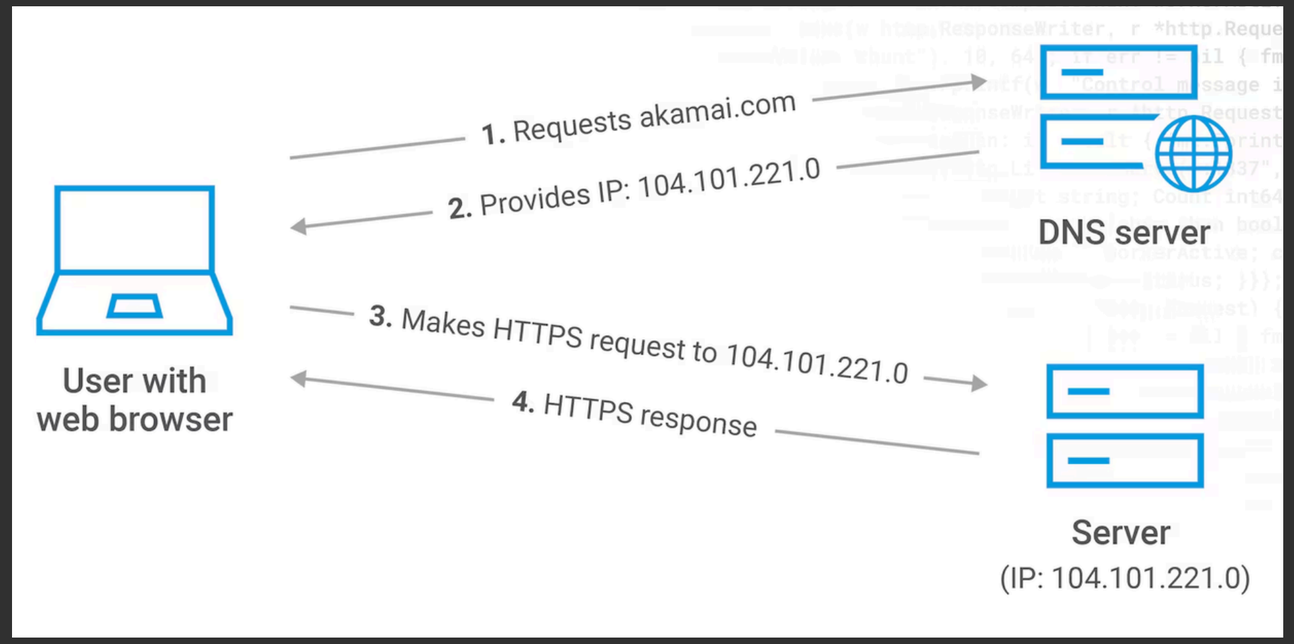
- Traduce dominio → dirección IP
- Permite acceder a sitios desde cualquier lugar
- Ej: google.com → IP del servidor

Scheme → protocolo (http/https)
Authority → dominio + puerto
Path → ubicación del recurso
Query → parámetros enviados al servidor
Fragment → sección específica del documento

PROCESO AL INGRESAR UNA URL

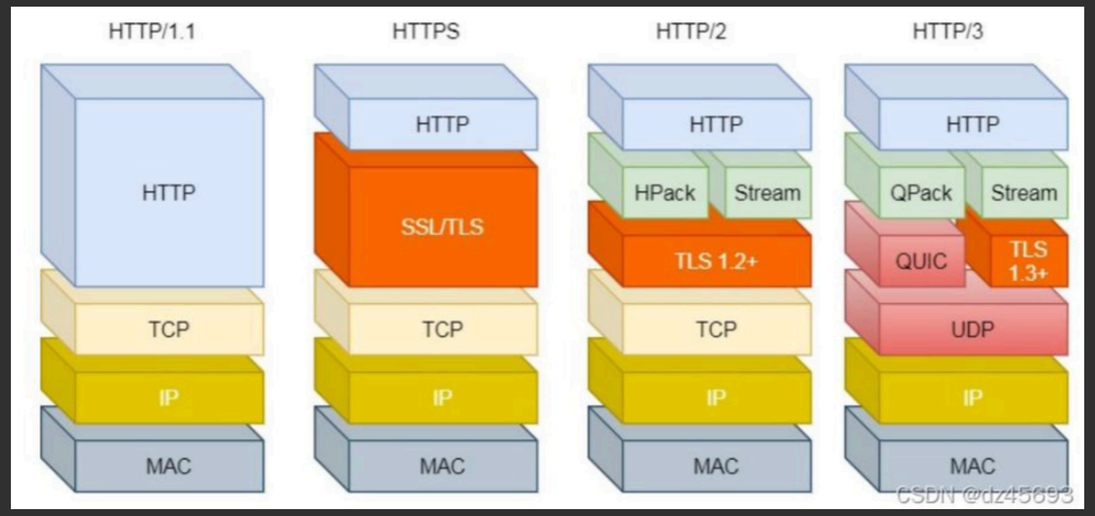
1. Browser consulta DNS → obtiene IP
2. Se establece conexión (TCP o QUIC)
3. Se envía HTTP Request
4. Servidor procesa y responde
5. Browser:

- Interpreta HTML
- Descarga CSS, JS, imágenes
- Renderiza la página



PROTOCOLO HTTP Y VERSIONES

HTTP = HyperText Transfer Protocol. Protocolo base de la Web.



HTTP/1.1:

- Basado en texto, Conexiones bloqueantes

HTTP/2:

- **Formato binario, Multiplexación, Compresión headers, Priorización:**

HTTP/3:

- Usa QUIC (UDP), Menor latencia, Mejor resiliencia

Multiplexación:
Múltiples requests en una conexión

Priorización:
Carga primero lo importante

Compresión de headers:
Reduce tamaño de datos

Formato binario:
Más eficiente que texto

- **HTTP/1.1:** Útil para aplicaciones o sistemas antiguos (*legacy*) simples que transmiten poco texto y no necesitan cargar múltiples recursos simultáneos de una página web moderna.
- **HTTP/2:** Ideal para la gran mayoría de sitios web modernos que cargan muchos recursos estáticos (múltiples imágenes, archivos CSS y JavaScript), ya que aprovecha la multiplexación para enviar todo por una sola conexión TCP.
- **HTTP/3:** Excelente para conexiones inestables (como redes móviles con mucha pérdida de señal) o para *streaming* de video y videojuegos en tiempo real. Al estar basado en QUIC y UDP, evita el problema del "bloqueo de cabecera" (*head-of-line blocking*), por lo que si un paquete se pierde, el resto de la página sigue cargando sin detenerse.

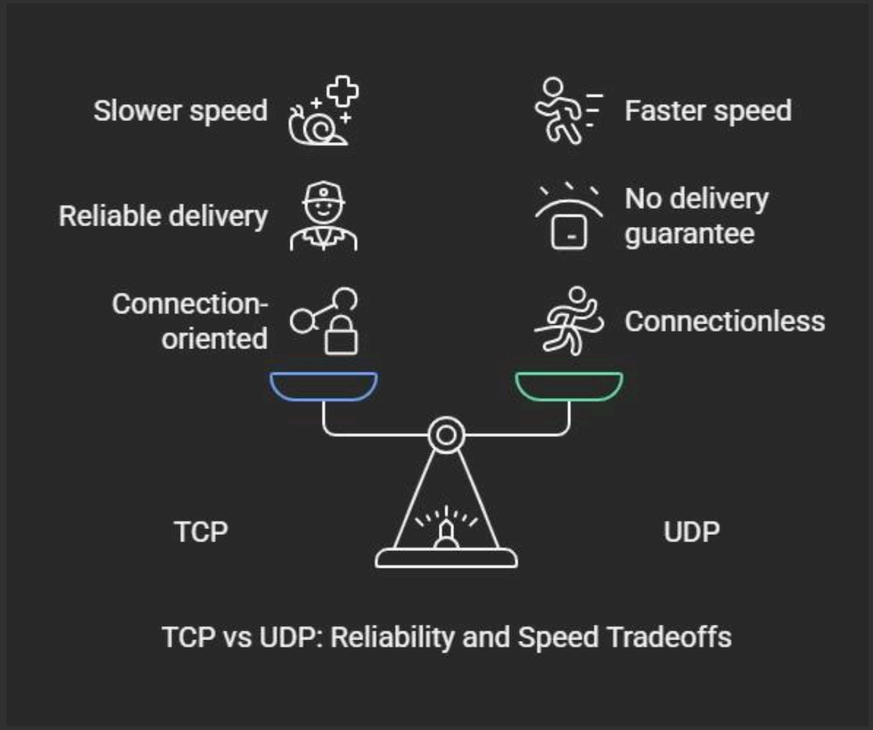
TCP, UDP Y SEGURIDAD

TCP:

- Protocolo confiable
- Paquetes enviados en secuencia
- Detecta y corrige errores
- Tiene control de congestión
- Usa acuse de recibo
- Permite establecer y mantener una conexión

TLS: Protocolo de seguridad

- Cifra la comunicación entre cliente y servidor
- Asegura confidencialidad e integridad de los datos
- Se usa junto a HTTP → HTTPS



UDP:

- Más rápido que TCP
- No garantiza entrega ni orden
- No tiene control de errores ni congestión
- Envío de datos como flujo (sin conexión)

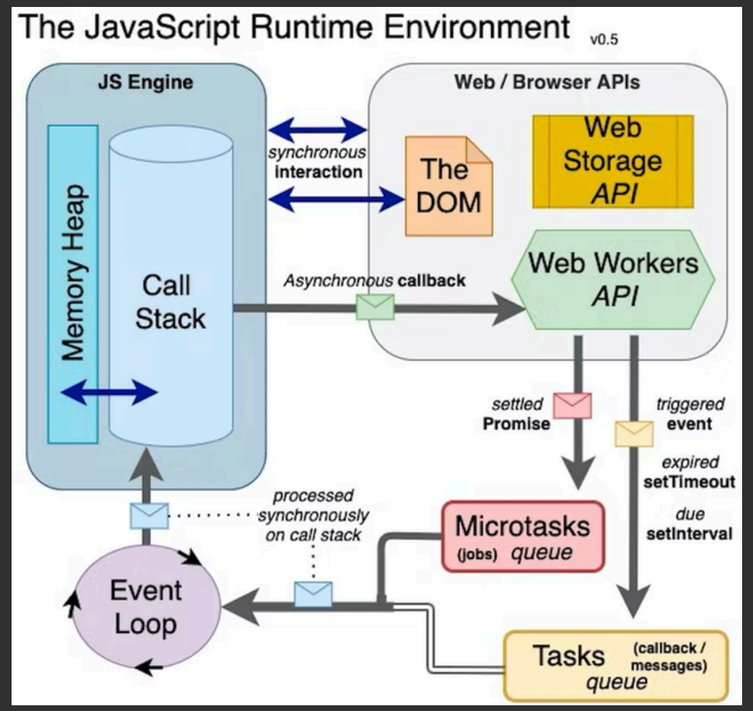
TCP + TLS + HTTP2: combinación tradicional para tener web rápida, confiable y segura.

JAVA SCRIPT

JavaScript, es un **lenguaje de scripting** que **corre sobre un engine (motor)** de browser (navegador) y **sigue los estándares** definidos por **ECMAScript.**

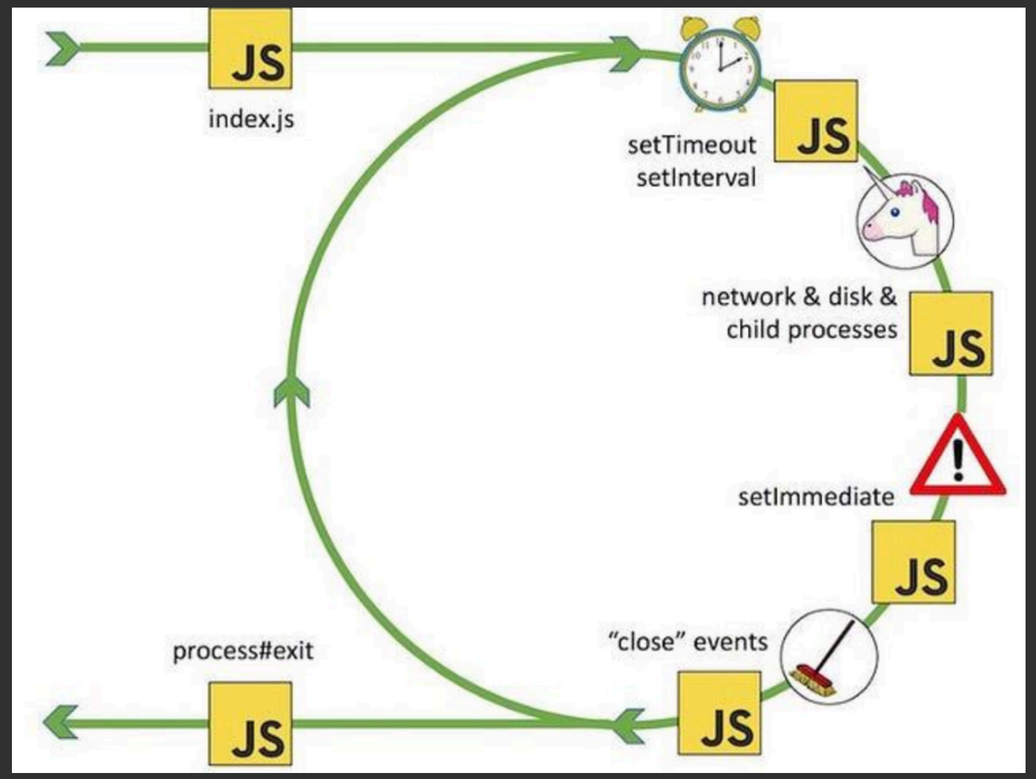
JavaScript corre sobre un runtime environment, y dentro de ese runtime está el engine que lo ejecuta. (V8 chrome)

runtime environment: hace que funcione en cualquier sistema operativo. Hay un rte para cada SO por eso al descargar browser te pregunta tu SO.



NODE JS

Plataforma opensource que corre en múltiples sistemas operativos que permite la ejecución de JavaScript fuera de un navegador.



El truco de Node.js para atender tantas solicitudes: **SINGLE THREAD**.

Single thread junto a una cola de mensajes y una arquitectura basada en un patrón de "event loop" que permite realizar operaciones non-blocking en I/O.

Tiene cola de eventos, los eventos faciles o comunes o rapidos se van ejecutando y despachando, cuando hay algún proceso largo o así, lo pone de lado como procesos con DB

EJERCICIOS

SELECCION SIMPLE

El Event Loop en Node.js permite:

a) Ejecutar múltiples threads en paralelo.
b) Atender operaciones I/O sin bloquear el flujo principal.
c) Convertir código síncrono en asíncrono automáticamente.
d) Evitar que se usen promesas o callbacks.

SELECCION SIMPLE

El Event Loop en Node.js permite:

a) Ejecutar múltiples threads en paralelo.
b) Atender operaciones I/O sin bloquear el flujo principal.
c) Convertir código síncrono en asíncrono automáticamente.
d) Evitar que se usen promesas o callbacks.

SELECCION SIMPLE

En HTTP, la principal diferencia entre 1.1 y 2 es:

a) HTTP/2 introduce compresión de cabeceras y multiplexación.
b) HTTP/2 elimina por completo el uso de TLS.
c) HTTP/1.1 usa UDP y HTTP/2 usa TCP.
d) HTTP/1.1 permite mayor seguridad que HTTP/2.

SELECCION SIMPLE

En HTTP, la principal diferencia entre 1.1 y 2 es:

a) HTTP/2 introduce compresión de cabeceras y multiplexación.
b) HTTP/2 elimina por completo el uso de TLS.
c) HTTP/1.1 usa UDP y HTTP/2 usa TCP.
d) HTTP/1.1 permite mayor seguridad que HTTP/2.

SELECCION SIMPLE

¿Qué describe mejor el rol de DNS en la Web?

a) Un protocolo de transporte confiable.
b) Un sistema que resuelve nombres de dominio a direcciones IP.
c) Un cortafuegos a nivel de aplicación.
d) Un mecanismo de compresión de cabeceras.

SELECCION SIMPLE

¿Qué describe mejor el rol de DNS en la Web?

a) Un protocolo de transporte confiable.
b) Un sistema que resuelve nombres de dominio a direcciones IP.
c) Un cortafuegos a nivel de aplicación.
d) Un mecanismo de compresión de cabeceras.

SELECCION SIMPLE

¿Qué es V8 en el contexto web/backend?

a) Un framework MVC.
b) El motor de JavaScript usado por Chrome y base de Node.js.
c) Un sistema de colas para I/O.
d) Un transpilador ES6.

SELECCION SIMPLE

¿Qué es V8 en el contexto web/backend?

a) Un framework MVC.
b) El motor de JavaScript usado por Chrome y base de Node.js.
c) Un sistema de colas para I/O.
d) Un transpilador ES6.

PREGUNTA HERENCIA/PROTOTIPOS

Tiene este código (subóptimo) donde cada instancia redefine métodos, duplicando memoria y dificultando el testeo:

```
function Libro(titulo, autor) {
  this.titulo = titulo;
  this.autor = autor;
  this.info = function () {
    return `${this.titulo} de ${this.autor}`;
  };
}


function Ebook(titulo, autor, formato) {
  Libro.call(this, titulo, autor);
  this.formato = formato;
  this.info = function () {
    return `${this.titulo} (${this.formato}) de
${this.autor}`;
  };
}
```

SE PIDE:

1. Explique por qué esto es ineficiente y cómo lo soluciona la herencia prototipada (cadena de prototipos.

Tiene este código (subóptimo) donde cada instancia redefine métodos, duplicando memoria y dificultando el testeo:

```
function Libro(titulo, autor) {
  this.titulo = titulo;
  this.autor = autor;
  this.info = function () {
    return `${this.titulo} de ${this.autor}`;
  };
}


function Ebook(titulo, autor, formato) {
  Libro.call(this, titulo, autor);
  this.formato = formato;
  this.info = function () {
    return `${this.titulo} (${this.formato}) de
${this.autor}`;
  };
}
```

SE PIDE:

1. Explique por qué esto es ineficiente y cómo lo soluciona la herencia prototipada (cadena de prototipos.

En este código, cada vez que se crea un nuevo Libro o Ebook, también se crea una nueva función info().

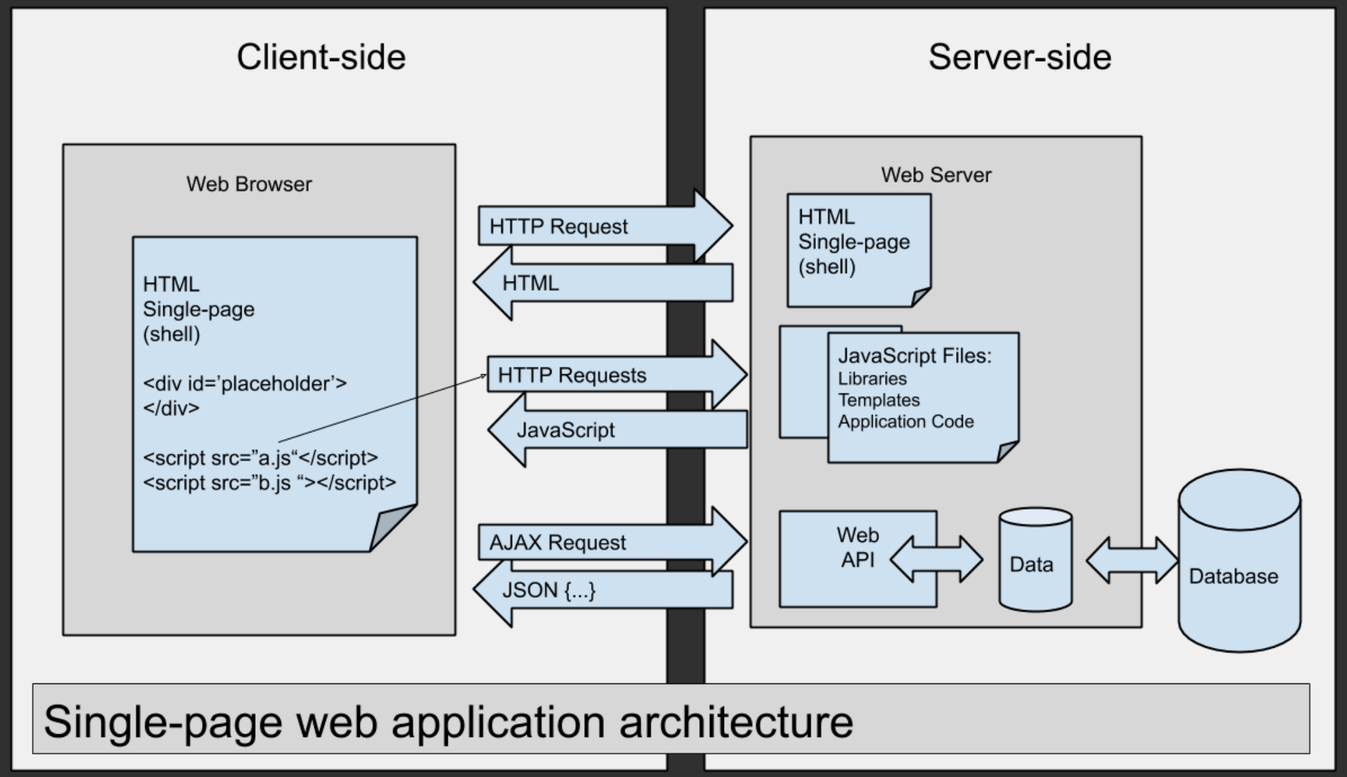
Esto es ineficiente porque todas las instancias contienen su propia copia del mismo método, ocupando memoria adicional y dificultando el mantenimiento o testeo.

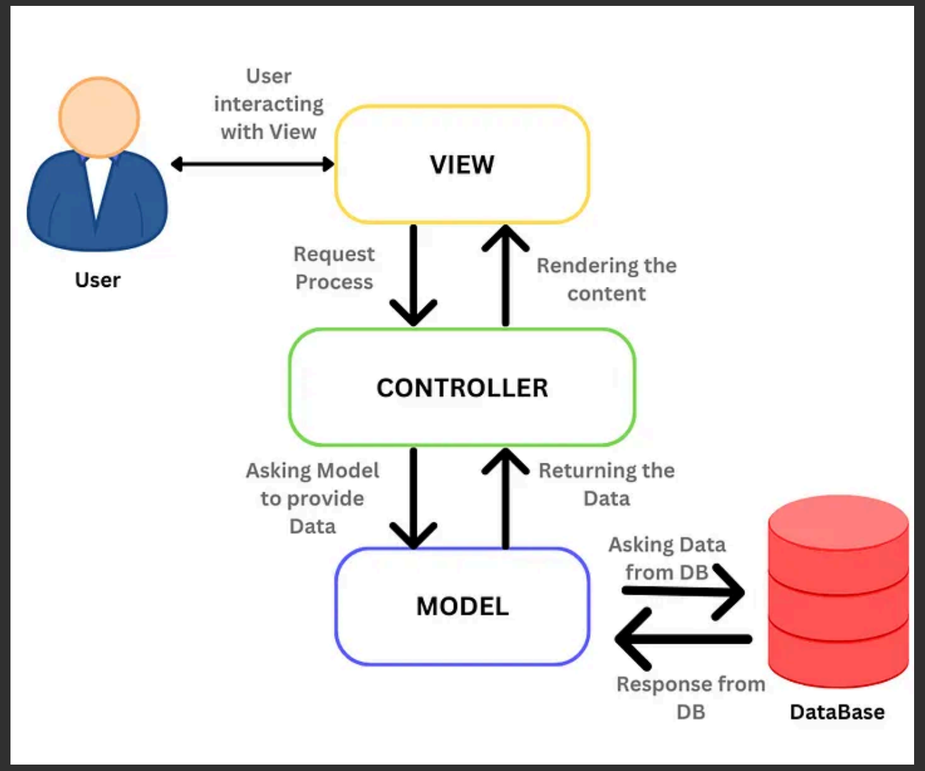
En JavaScript, el mecanismo de herencia prototipada permite que los métodos se definan una sola vez en el objeto prototipo, de modo que todas las instancias los compartan a través de la cadena de prototipos.

ebook
↓
Ebook.prototype
↓
Libro.prototype
↓
Object.prototype
↓
null

Cada instancia (ebook) no tiene sus propios métodos: los busca en Ebook.prototype, y si no están, sube la cadena hasta Libro.prototype y finalmente Object.prototype.

ESQUEMA DE APLICACIÓN WEB





AYUDANTÍA I1

JUAN JOSÉ MERUANE
ARTURO PACHECO